

Caching Fundamentals

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

English:

A cache is a small, fast storage layer that keeps a copy of frequently-needed data so future requests can be served without repeating expensive work. A well-placed cache can turn a 200ms database query into a 2ms lookup.

Hindi:

एक cache एक छोटी, तेज़ storage layer होती है जो बार-बार चाहिए वाले data की एक copy रखती है ताकि आगे आने वाली requests को महंगा काम दोबारा किए बिना पूरा किया जा सके। सही जगह लगाई गई cache एक 200ms वाली database query को 2ms के lookup में बदल सकती है।

Backend engineer analogy

English:

You've done this exact thing with Laravel's `Cache::remember()` wrapping an expensive query, or Redis storing a rendered API response. System design just asks: where should this cache live, and what happens when the underlying data changes?

Hindi:

आपने यह बिल्कुल यही काम किया है Laravel के `Cache::remember()` से किसी महंगी query को wrap करके, या Redis में किसी rendered API response को store करके। System design बस यह पूछता है: यह cache कहाँ रहनी चाहिए, और जब underlying data बदल जाए तो क्या होगा?

Important Words:

- **Underlying** – अंतर्गति / मूल

The three core caching patterns

English:

- **Cache-Aside (Lazy Loading):** the application checks the cache first; on a miss, it reads from the database and then writes the result into the cache.
- **Write-Through:** every write goes to the cache and the database together, synchronously.
- **Write-Back (Write-Behind):** writes go to the cache first and are flushed to the database asynchronously later.

Hindi:

- **Cache-Aside (Lazy Loading):** application पहले cache check करता है; miss होने पर database से पढ़ता है और फिर result को cache में लिख देता है।
- **Write-Through:** हर write एक साथ cache और database दोनों में synchronously जाता है।

- Write-Back (Write-Behind): writes पहले cache में जाते हैं और बाद में asynchronously database में flush होते हैं।

Important Words:

- **Cache miss** – कैश मिस
- **Flush (verb)** – फ्लश करना / भेज देना

Eviction: what happens when the cache is full

English:

A cache has finite memory, so it needs a policy for what to discard when full. LRU (Least Recently Used) is the most common default -- evict whatever hasn't been accessed in the longest time.

Hindi:

एक cache की memory सीमति होती है, इसलिए जब वह भर जाए तो कुछ ना कुछ हटाने की policy चाहिए होती है। LRU (Least Recently Used) सबसे आम default है -- जो सबसे लंबे समय से access नहीं हुआ, उसे हटा दिया जाता है।

Important Words:

- **Evict** – हटाना / बाहर निकालना

English:

Caching's fundamental risk is serving outdated data after the underlying record changes. The usual fixes are setting a sensible TTL or actively invalidating the cache entry the moment the underlying data changes.

Hindi:

Caching का सबसे बड़ा risk यह है कि underlying data बदलने के बाद भी पुराना data दखिता रहे। इसका आम हल है एक सही TTL सेट करना, या underlying data बदलते ही उस cache entry को actively invalidate कर देना।

Important Words:

- **Invalidate** – अमान्य करना / हटा देना

English:

It's also worth distinguishing where a cache can live. An in-process cache is fastest but only visible to that one server instance. A shared cache like Redis is slightly slower per-lookup but consistent across your whole fleet.

Hindi:

यह समझना भी ज़रूरी है कि cache कहाँ रह सकती है। एक in-process cache सबसे तेज़ होती है लेकिन सिर्फ उस एक server instance को दखित है। Redis जैसी shared cache हर lookup में थोड़ी धीमी होती है लेकिन पूरे fleet में एक जैसी (consistent) रहती है।

English:

One last practical point: caching only helps if the same data is requested repeatedly. Before adding a cache, it's worth confirming there actually is a 'hot' subset of data or queries being repeated.

Hindi:

एक आखिरी practical बात: caching तभी काम आती है जब एक ही data बार-बार मांगा जाए। cache जोड़ने से पहले, यह पक्का कर लेना ज़रूरी है कि सिच में कोई 'hot' data या repeated queries मौजूद हैं।

Important Words:

- **Hot (data, figurative)** – बार-बार इस्तेमाल होने वाला

2. Real-World Example / Case Study

English:

PhonePe and Razorpay-style payment platforms cache things like merchant configuration and rate-limit counters in Redis because these are read constantly but change rarely -- a perfect cache-aside candidate.

Hindi:

PhonePe और Razorpay जैसे payment platforms merchant configuration और rate-limit counters जैसी चीज़ें Redis में cache करते हैं क्योंकि यह लगातार पढ़ी जाती है लेकिन कम ही बदलती है -- यह cache-aside के लिए एक बिल्कुल सही उदाहरण है।

English:

A frequently cited real incident pattern is cache stampede: an extremely popular cache key expires, and thousands of concurrent requests all miss the cache at once and hammer the database simultaneously.

Hindi:

एक अक्सर बताया जाने वाला असली incident pattern है cache stampede: एक बहुत popular cache key expire हो जाती है, और हजारों requests एक साथ cache miss करके database पर एक साथ टूट पड़ती है।

Important Words:

- **Hammer (verb, figurative)** – बार-बार ज़ोर से हटि करना

English:

A second example: Meesho caches category-level product listings aggressively since these change infrequently but are read by nearly every user -- an extremely favorable read:write ratio.

Hindi:

एक और उदाहरण: Meesho category-level product listings को काफी aggressively cache करता है क्योंकि यह कम ही बदलती है लेकिन लगभग हर user इन्हें पढ़ता है -- यह एक बहुत अच्छा read:write ratio है।

Important Words:

- **Ratio** – अनुपात

3. Diagram (English labels kept as-is)

Cache-Aside (most common pattern):

```
App --- 1. GET product:123 --> Cache
App <-- 2. MISS ----- Cache
App --- 3. SELECT ... WHERE id=123 --> Database
App <-- 4. row returned ----- Database
App --- 5. SET product:123 = row, TTL 300s -----> Cache

Next request within 300s:
App --- GET product:123 --> Cache
App <-- HIT (fast!) ----- Cache
```

4. Trade-offs Table (Reference Table – English)

Pattern	Read speed after warm-up	Write consistency	Risk
Cache-Aside	Fast	Can go stale until TTL/invalidation	Cache stampede on hot key expiry
Write-Through	Fast, always fresh	Strong	Every write pays double latency
Write-Back	Fast reads and writes	Weak	Data loss if cache crashes before flush

5. Common Interview Questions

Q1: "How would you keep a cache from serving stale data?"

English:

Model approach: Offer two levers -- a bounded TTL (simple, works for most read-heavy data) or active invalidation (better freshness but more moving parts).

Hindi:

Model approach: दो तरीके बताएं -- एक bounded TTL (सरल, ज्यादातर read-heavy data के लिए काम करता है) या active invalidation (ज्यादा ताज़ा data देता है लेकिन थोड़ा जटिल भी)।

Q2: "A flash-sale page just crashed your database -- what happened, and how do you prevent it?"

English:

Model approach: Name cache stampede explicitly -- a hot key expired and thousands of requests hit the DB at once. Propose a locking mechanism or pre-warming the cache before the sale starts.

Hindi:

Model approach: सीधे cache stampede का नाम लें -- एक hot key expire हुई और हजारों requests एक साथ DB पर गईं। एक locking mechanism या sale शुरू होने से पहले cache को pre-warm करने का सुझाव दें।

Important Words:

- **Pre-warming** – पहले से भर देना

Q3: "When would an in-process (local) cache still make sense?"

English:

Model approach: For extremely hot, tiny, tolerant-of-inconsistency data, a local in-process cache avoids even the small network hop to Redis, layered as an L1/L2 combination.

Hindi:

Model approach: बहुत ज्यादा hot, छोटे, और थोड़ी inconsistency सहन कर सकने वाले data के लिए, एक local in-process cache Redis तक जाने वाला छोटा network hop भी बचा देती है, इसे L1/L2 combination की तरह use करें।

6. Practice Exercises

English:

- For a news-feed 'like count,' decide between cache-aside, write-through, and write-back, and justify it.
- Explain cache stampede using a non-technical analogy.
- Design the cache key naming scheme for a product catalog cache.

Hindi:

- एक news-feed के 'like count' के लिए cache-aside, write-through, और write-back में से चुने और वजह बताएं।
- cache stampede को किसी non-technical analogy से समझाएं।
- एक product catalog cache के लिए cache key naming scheme design करें।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"Redis Caching Strategies Explained: Cache Aside vs Write-Through"	Directly compares the patterns.
English	"Caching Explained: Redis, Cache-Aside, & LRU"	Ties caching to interview framing.
Hindi	Redis explained in Hinglish (caching, rate limiting, sessions, OTP)	Practical use cases.
Hindi	"Redis Full Course 2026" — Chai aur Code	Full Hindi course on caching patterns.

8. Key Takeaways

English:

- Caching stores frequently-needed data in a fast layer to avoid repeating expensive work.
- Cache-aside is the default pattern: check cache, on miss read DB and populate cache.
- Write-through keeps cache and DB always in sync but slows writes; write-back is fast but risky.
- LRU is the most common eviction policy for a full cache.
- Every cache introduces a staleness trade-off you must explicitly decide on.
- Cache stampede is a classic real-world failure and a strong interview talking point.

Hindi:

- Caching बार-बार चाहिए वाले data को एक तेज़ layer में रखती है ताकि भिहंगा काम दोबारा ना करना पड़े।
- Cache-aside default pattern है: पहले cache check करें, miss होने पर DB पढ़ें और cache भर दें।
- Write-through cache और DB को हमेशा sync रखता है लेकिन writes धीमी होती है; write-back तेज़ पर risky है।
- पूरी हो चुकी cache के लिए LRU सबसे आम eviction policy है।
- हर cache के साथ staleness का trade-off आता है, जसै आपको खुद तय करना होता है।
- Cache stampede एक classic असली failure है और interview में बताने के लिए एक मज़बूत उदाहरण है।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Cache miss	कैश मिस	When requested data is not found in the cache
Evict	हटाना / बाहर निकालना	To remove something to make space
Flush (verb)	फ्लश करना / भेज देना	To write pending data out to permanent storage
Hammer (verb, figurative)	बार-बार ज़ोर से हटि करना	To hit something repeatedly and heavily
Hot (data, figurative)	बार-बार इस्तेमाल होने वाला	Frequently accessed
Invalidate	अमान्य करना / हटा देना	To mark old cached data as no longer valid
Pre-warming	पहले से भर देना	Loading the cache with data before it's needed
Ratio	अनुपात	A comparison between two quantities
Underlying	अंतर्निहित / मूल	The original, base data behind something